

DQuez Carr

Module 1: Lab Exercise

Orientation to Virtual Machine and Lab Environment

Short description of methods used to access the virtual machines.

I accessed the VM by first downloading and installing the NKU VPN. Afterwards, I followed the sequence of steps to access the COI VM Host and logged into the Kali, Meta, and Pfsense VM's.

Working on Kali Linux

- Commands, interpretations of outputs, and cropped screenshots (as presented in the lab instructions)

```
student@mcy670-kali:~$ sudo -s
[sudo] password for student:
root@mcy670-kali:/home/student# exit
exit
student@mcy670-kali:~$ pwd
/home/student
student@mcy670-kali:~$ cd /
student@mcy670-kali:/$ cd /home/
student@mcy670-kali:/home$ cd /home/student
student@mcy670-kali:~$ cd /home/student/
student@mcy670-kali:~$ cd
student@mcy670-kali:~$ pwd
/home/student
student@mcy670-kali:~$ ls
armitage-tmp  Desktop  Downloads  nessus_rest.rb  Pictures  Templates
change-ip.sh  Documents Music        nessus_rest.rb.bkup  Public  Videos
student@mcy670-kali:~$ ls -l
armitage-tmp
change-ip.sh
Desktop
Documents
Downloads
Music
nessus_rest.rb
nessus_rest.rb.bkup
Pictures
Public
Templates
Videos
student@mcy670-kali:~$
```

```
student@mcy670-kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1450
    inet 192.168.1.202 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::be24:11ff:fe75:eaf9 prefixlen 64 scopeid 0x20<link>
    ether bc:24:11:75:ea:f9 txqueuelen 1000 (Ethernet)
    RX packets 8 bytes 1018 (1018.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 11 bytes 1129 (1.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 14 bytes 718 (718.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14 bytes 718 (718.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
student@mcy670-kali:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1450
    inet 192.168.1.202 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::be24:11ff:fe75:eaf9 prefixlen 64 scopeid 0<link>
    ether bc:24:11:75:ea:f9 txqueuelen 1000 (Ethernet)
    RX packets 8 bytes 1018 (1018.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 11 bytes 1129 (1.1 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 14 bytes 718 (718.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14 bytes 718 (718.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr bc:24:11:45:6b:0e
          inet addr:192.168.1.203 Bcast:192.168.1.255 Mask:255.255.255.0
          inet6 addr: fe80::be24:11ff:fe45:6b0e/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:38 errors:0 dropped:0 overruns:0 frame:0
          TX packets:64 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3547 (3.4 KB)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:98 errors:0 dropped:0 overruns:0 frame:0
          TX packets:98 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:21621 (21.1 KB)  TX bytes:21621 (21.1 KB)

msfadmin@metasploitable:~$ _
```

Verifying VM IP Addresses

META VM

- Cropped screenshot of output for successfully pinging your KALI VM IP address from META.

```
msfadmin@metasploitable:~$ ping 192.168.1.202
PING 192.168.1.202 (192.168.1.202) 56(84) bytes of data:
64 bytes from 192.168.1.202: icmp_seq=1 ttl=64 time=6.39 ms
64 bytes from 192.168.1.202: icmp_seq=2 ttl=64 time=0.378 ms
64 bytes from 192.168.1.202: icmp_seq=3 ttl=64 time=0.367 ms
64 bytes from 192.168.1.202: icmp_seq=4 ttl=64 time=0.383 ms
64 bytes from 192.168.1.202: icmp_seq=5 ttl=64 time=0.378 ms
64 bytes from 192.168.1.202: icmp_seq=6 ttl=64 time=0.357 ms
64 bytes from 192.168.1.202: icmp_seq=7 ttl=64 time=0.357 ms

--- 192.168.1.202 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6008ms
rtt min/avg/max/mdev = 0.357/1.231/6.399/2.109 ms
msfadmin@metasploitable:~$
```

- Cropped screenshot of output for successfully pinging google(.)com

```
msfadmin@metasploitable:~$ ping 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=47 time=14.5 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=47 time=14.4 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=47 time=14.3 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=47 time=14.4 ms
64 bytes from 1.1.1.1: icmp_seq=5 ttl=47 time=14.4 ms

--- 1.1.1.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4003ms
rtt min/avg/max/mdev = 14.337/14.440/14.535/0.166 ms
msfadmin@metasploitable:~$
```

KALI VM

- Cropped screenshot of output for successfully pinging your META VM IP address from KALI.

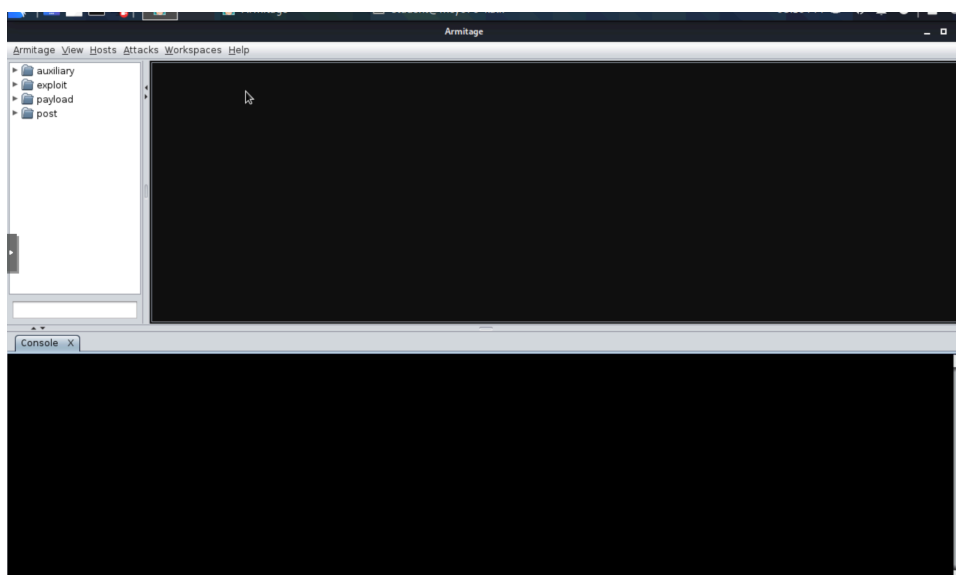
```
student@mcy670-kali:~$ ping 192.168.1.203
PING 192.168.1.203 (192.168.1.203) 56(84) bytes of data.
64 bytes from 192.168.1.203: icmp_seq=1 ttl=64 time=0.368 ms
64 bytes from 192.168.1.203: icmp_seq=2 ttl=64 time=0.410 ms
64 bytes from 192.168.1.203: icmp_seq=3 ttl=64 time=0.349 ms
64 bytes from 192.168.1.203: icmp_seq=4 ttl=64 time=0.347 ms
64 bytes from 192.168.1.203: icmp_seq=5 ttl=64 time=0.397 ms
64 bytes from 192.168.1.203: icmp_seq=6 ttl=64 time=0.353 ms
64 bytes from 192.168.1.203: icmp_seq=7 ttl=64 time=0.372 ms
64 bytes from 192.168.1.203: icmp_seq=8 ttl=64 time=0.347 ms
^C
--- 192.168.1.203 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7153ms
rtt min/avg/max/mdev = 0.347/0.367/0.410/0.022 ms
student@mcy670-kali:~$
```

- Cropped screenshot of output for successfully pinging google(.)com

```
student@mcy670-kali:~$ ping 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=47 time=14.5 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=47 time=14.3 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=47 time=14.3 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=47 time=14.3 ms
64 bytes from 1.1.1.1: icmp_seq=5 ttl=47 time=14.3 ms
64 bytes from 1.1.1.1: icmp_seq=6 ttl=47 time=14.1 ms
64 bytes from 1.1.1.1: icmp_seq=7 ttl=47 time=14.5 ms
64 bytes from 1.1.1.1: icmp_seq=8 ttl=47 time=14.4 ms
64 bytes from 1.1.1.1: icmp_seq=9 ttl=47 time=14.4 ms
64 bytes from 1.1.1.1: icmp_seq=10 ttl=47 time=14.3 ms
^C
--- 1.1.1.1 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9013ms
rtt min/avg/max/mdev = 14.124/14.336/14.487/0.098 ms
student@mcy670-kali:~$
```

Running msfconsole on Kali Linux VM

- Short text description of each command and interpretation of outputs
I first ensured that postgresql was running and my database was properly configured. After that, I started msfconsole and verified database connectivity.
- Cropped screenshots for successful database initialization, launch of msfconsole, and verifying database connection



MSFVenom tool on Kali Linux VM

- Short text description of each command and interpretation of output
- Cropped screenshots of msfvenom example payload options, list of payloads, encoders, and architectures

```
student@ncy670-kali:~$ msfvenom -h
MsfVenom - a Metasploit standalone payload generator.
Also a replacement for msfpayload and msfencode.
Usage: /usr/bin/msfvenom [options] <var-val>
Example: /usr/bin/msfvenom -p windows/meterpreter/reverse_tcp LHOST=<IP> -f exe -o payload.exe

Options:
  -l, --list <type>      List all modules for [type]. Types are: payloads, encoders, nops, platforms, archs,
                        encrypt, formats, all
  -p, --payload <payload> Payload to use (--list payloads to list, --list-options for arguments). Specify '-'
                        or STDIN for custom
                        --list-options List --payload <value>'s standard, advanced and evasion options
  -f, --format <format>  Output format (use --list formats to list)
  -e, --encoder <encoder> The encoder to use (use --list encoders to list)
                        --service-name <value> The service name to use when generating a service binary
                        --sec-name <value> The new section name to use when generating large Windows binaries. Default: random
  4-character alpha string
  --smallest <value> Generate the smallest possible payload using all available encoders
  --encrypt <value> The type of encryption or encoding to apply to the shellcode (use --list encrypt to
list)
  --encrypt-key <value> A key to be used for --encrypt
  --encrypt-iv <value> An initialization vector for --encrypt
  -a, --arch <arch> The architecture to use for --payload and --encoders (use --list archs to list)
  --platform <platform> The platform for --payload (use --list platforms to list)
  -o, --out <path> Save the payload to a file
  -b, --bad-chars <list> Characters to avoid example: '\x00\xff'
  -n, --nopsled <length> Prepend a nopsled of [length] size on to the payload
                        --pad-nops <length> Use nopsled size specified by -n <length> as the total payload size, auto-prepend
a nopsled of quantity (nops minus payload length)
  -s, --space <length> The maximum size of the resulting payload
  --encoder-space <length> The maximum size of the encoded payload (defaults to the -s value)
  -i, --iterations <count> The number of times to encode the payload
  -c, --add-code <path> Specify an additional win32 shellcode file to include
  -x, --template <path> Specify a custom executable file to use as a template
```

```
student@mcy670-kali:~$ msfvenom -p windows/shell_reverse_tcp --list-options
Options for payload/windows/shell_reverse_tcp:

Name: Windows Command Shell, Reverse TCP Inline
Module: payload/windows/shell_reverse_tcp
Platform: Windows
Arch: x86
Needs Admin: No
Total size: 324
Rank: Normal

Provided by:
vlad902 <vlad902@gmail.com>
sf <stephen_fewer@harmonysecurity.com>

Basic options:
Name      Current Setting  Required  Description
-----
EXITFUNC  process          yes       Exit technique (Accepted: '', seh, thread, process, none)
LHOST     4444             yes       The listen address (an interface may be specified)
LPORT     4444             yes       The listen port

Description:
Connect back to attacker and spawn a command shell

Advanced options for payload/windows/shell_reverse_tcp:

Name      Current Setting  Required  Description
-----
AutoRunScript      no       A script to run automatically on session creation.
CommandShellCleanupCommand  no       A command to run before the session is closed
CreateSession      true      Create a new session for every successful login
InitialAutoRunScript  no       An initial script to run on session creation (before AutoRunSc
ript)
PrependMigrate      false     Spawns and runs shellcode in new process
PrependMigrateProc  no       Process to spawn and run shellcode in
ReverseAllowProxy    false     Allow reverse tcp even with Proxies specified. Connect back wi
ll NOT go through proxy but directly to LHOST
ReverseListenerBindAddress  no       The specific IP address to bind to on the local system
ReverseListenerBindPort    no       The port to bind to on the local system if different from LPOR
T
ReverseListenerComm  no       The specific communication channel to use for this listener
ReverseListenerThreaded  false     Handle every connection in a new thread (experimental)
StagerRetryCount     10       The number of times the stager should retry if the first conne
ct fails
StagerRetryWait      5        Number of seconds to wait for the stager between reconnect att
empts
VERBOSE              false     Enable detailed status messages
WORKSPACE             no       Specify the workspace for this module

Evasion options for payload/windows/shell_reverse_tcp:

Name      Current Setting  Required  Description
-----
student@mcy670-kali:~$
```

NASM tool on Kali Linux VM

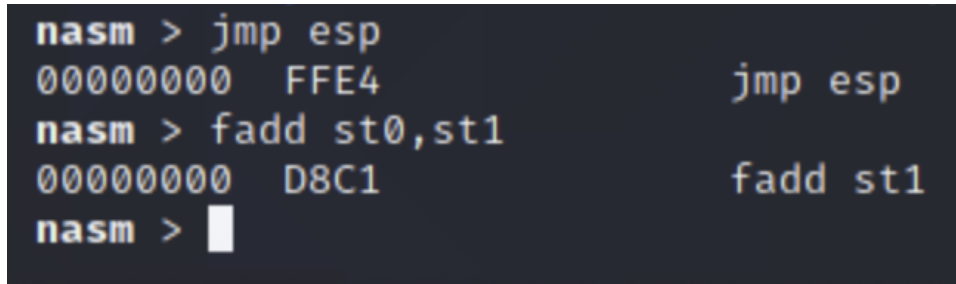
- Cropped screenshots of successful launch of nasm

```
student@mcy670-kali:~$ cd /
student@mcy670-kali:/$ locate nasm_shell
/usr/bin/msf-nasm_shell
/usr/share/metasploit-framework/tools/exploit/nasm_shell.rb
student@mcy670-kali:/$
::1      ff02::2      ip6-allrouters  ip6-loopback  mcy670-kali
ff02::1  ip6-allnodes ip6-localhost   localhost
student@mcy670-kali:/$ cd
::1      ff02::2      ip6-allrouters  ip6-loopback  mcy670-kali
ff02::1  ip6-allnodes ip6-localhost   localhost
student@mcy670-kali:/$ cd /usr/share/metasploit-framework/tools/exploit/
student@mcy670-kali:/usr/share/metasploit-framework/tools/exploit$ ./nasm_shell.rb
nasm >
```

- Short text description of each command and interpretation of outputs

I first located the utility nasm by navigating to the root directory using `cd /`, then I searched all directories by using the `locate` command on the file name `nasm_shell`. I then went to that folder and launched the tool using a script. After launching the tool i requested opcodes for the below assembly language instructions.

- Cropped screenshots of sample command outputs



```
nasm > jmp esp
00000000 FFE4 jmp esp
nasm > fadd st0,st1
00000000 D8C1 fadd st1
nasm > 
```

Brief reflection of the lab exercise:

- What was the most valuable feature of the lab?
I believe the most valuable feature of this lab is the configuration and understanding of Metasploit through Kali linux vm. Understanding the capabilities and limitations of this application will be key to excelling in this course.
- What did you learn from this experience?
I learned about the application Armitage which is a graphical interactive tool for Metasploit. This tool is extremely useful from a user's point of view. I hope to use the tool a lot more to understand the capabilities and limitations of the tool.
- Which concepts and terminology could you relate from the theoretical context to this lab experience?
I would be inclined to include learning the tools Nmap or Zenmap, OWASP ZAP, and Wireshark alongside the tools presented in this lab. These are needed to understand so that one can do the scanning step prior to exploiting any found vulnerabilities.
- How did you prepare for this lab?
I prepared for this lab by going through the lecture material and briefing myself on Kali linux. I've used Kali Linux before in past courses and home labs.
- What changes are you considering in preparing for your next lab?
I intend to start my Labs soon so that I am able to experiment with the tools more.